

A Lineage-Centered Survey of Landmark Optimizers: SGD, Adam, AdamW, SignSGD, Shampoo, and Muon (with MuP Context)

Abstract

Modern deep learning practice relies on stochastic, first-order optimization methods whose design balances compute, memory, stability, and hyperparameter sensitivity. This survey traces a lineage from classical stochastic gradient descent (SGD) and momentum-style smoothing to adaptive moment and preconditioning methods, and then to communication-efficient sign updates and scaling-oriented parameterizations. Adam introduces per-parameter adaptive steps via exponential moving averages of first and second gradient moments, with bias correction and update magnitudes that are invariant to gradient rescaling and approximately bounded by the step-size hyperparameter. [1] AdamW revisits regularization, arguing that coupling L_2 regularization to adaptive gradient scaling is undesirable, and advocating a decoupled weight decay step to reduce implicit hyperparameter coupling. [2] SignSGD studies the extreme quantization of updates to one bit, providing non-convex convergence guarantees under assumptions tailored to signed updates and extending the framework to a momentum-sign variant (SIGNUM) with 1-bit distributed majority-vote communication. [3] Shampoo develops structured second-order preconditioning using Kronecker-factored accumulators with regret bounds for matrix/tensor parameters. [4] Muon proposes momentum plus iterative orthogonalization of gradient matrices for hidden-layer weights. [5] Finally, MuP is included as context: it is not an optimizer, but a parameterization intended to make hyperparameters (including learning rates) more transferable across width scaling. [6]

Contents

1	Introduction	2
2	Notation	2
3	Lineage Map (Core Section)	3
3.1	Textual lineage overview	3
3.2	Per-optimizer lineage cards	3
4	Optimizer Families	5
4.1	SGD & Momentum	5
4.2	Adaptive methods	5
4.3	Second-order / preconditioning	5
4.4	Sign-based / communication-efficient methods	5
4.5	Specialized scaling- or structure-oriented methods	6
5	Comparative Analysis	6
5.1	Side-by-side update equations (consistent notation)	6
5.2	Table 1: Mechanism-level comparison	8

5.3	Table 2: Practical tradeoffs	9
5.4	Failure modes and “performs poorly” contexts	9
6	Research Insights & Open Questions	10
6.1	Synthesized insights (grounded in provided sources)	10
6.2	Open questions (motivated directly by provided sources)	11
7	Conclusion	12
A	Update-rule cheat sheet	12
B	Glossary of terms	13

1 Introduction

Optimizer design matters because training dynamics are shaped not only by the loss landscape and stochastic gradients, but also by how updates are scaled, regularized, and communicated. [1, 4, 3] The provided sources motivate optimizers that trade off (i) compute and memory overhead (e.g., storing moment vectors vs. matrix preconditioners), [1, 4] (ii) stability properties such as bounded or normalized update magnitudes and robustness to sparse gradients, [1] (iii) hyperparameter coupling and sensitivity (notably between learning rate and weight decay/regularization), [2] (iv) communication efficiency for distributed training (e.g., 1-bit sign transmission), [3] and (v) scaling behavior when changing model width, where MuP aims to preserve training dynamics under width scaling. [6]

Throughout, we compare methods along the axes requested: compute per step, additional state/memory, stability and failure modes, hyperparameter sensitivity and coupling, and scaling-oriented considerations. [1, 2, 3, 4, 6]

2 Notation

We follow the stochastic optimization setup used by the provided papers: parameters are updated iteratively using (stochastic) gradients of per-step objectives. [1]

Symbol	Meaning
t	iteration index
θ_t	parameter vector at iteration t
$f_t(\theta)$	stochastic objective (or loss) realization at iteration t
$g_t = \nabla_{\theta} f_t(\theta_{t-1})$	stochastic gradient
α	learning rate / stepsize
λ	weight decay or L_2 regularization factor (context-dependent)
m_t, v_t	exponential moving averages (first/second moments)
ϵ	numerical stabilizer

Table 1: Core notation used across optimizers (consistent with the algorithmic setups in the provided sources). [1, 2]

3 Lineage Map (Core Section)

3.1 Textual lineage overview

SGD $\rightarrow$ momentum-style smoothing. The provided sources treat SGD (and momentum variants) as a baseline and point of comparison in experiments and discussion. [1, 2, 3] In Adam and SignSGD, momentum is framed as maintaining a running average of recent gradients and using that average to define steps (directly or through a sign operator). [1, 3] Precise historical attributions (e.g., Polyak vs. Nesterov) are *not specified in the provided sources*.

AdaGrad/RMSProp $\rightarrow$ Adam $\rightarrow$ AdamW. Adam is explicitly designed to combine advantages of AdaGrad (sparse-gradient behavior) and RMSProp (non-stationary/online settings), by using exponential moving averages of gradients and squared gradients to form adaptive per-parameter learning rates. [1] AdamW argues that the common practice of implementing “weight decay” as an L_2 penalty inside the gradient update introduces undesirable coupling with the learning rate, and further that adaptive gradient scaling breaks the equivalence between L_2 regularization and decoupled weight decay; it therefore advocates a separate weight decay step. [2]

Sign-based lineage: (S)GD $\rightarrow$ SignSGD $\rightarrow$ SIGNUM (sign of momentum). SignSGD studies updates that keep only the sign of gradient components, motivated by gradient compression and by empirical success of sign-based methods. [3] The paper extends its theory to SIGNUM, defined as taking the sign of a momentum accumulator, and provides convergence rates under its framework. [3]

Structured preconditioning: OMD/preconditioning view $\rightarrow$ Shampoo. Shampoo frames its update as a preconditioned gradient step, derived from online mirror descent with time-varying regularization, and develops Kronecker-structured preconditioners for matrix/tensor parameters with regret guarantees. [4]

Orthogonalized updates: (matrix) preconditioning ideas $\rightarrow$ Muon. Muon proposes an update in which (for 2D parameter matrices) the gradient is iteratively transformed toward a semi-orthogonal matrix via a Newton–Schulz-based procedure, then used in a momentum update. [5] The source positions Muon as a method targeted to hidden-layer weight matrices and describes practical integration with AdamW for other parameter types. [5]

Scaling context: MuP (not an optimizer). MuP is presented as a parameterization intended to enable hyperparameter transfer across width scaling, and includes learning-rate stability analyses for optimizers such as SGD and Adam under specific parameterizations. [6] It is not itself an optimizer in the sense of defining a new update rule, but it changes how parameters are scaled so that standard optimizers behave consistently across width. [6]

3.2 Per-optimizer lineage cards

SGD. Core idea: iterative updates along (stochastic) negative gradients, used as a baseline and comparator throughout the provided sources. [1, 3, 2] What it borrowed: *Not specified in the provided sources*. What it changed/added: momentum variants are described as using a running average of recent gradients. [3, 2] Why it mattered historically: it is treated as the reference point

for convergence rate comparisons and as a foundation for hyperparameter “lifting” in SIGNUM experiments. [3]

Adam. Core idea: exponential moving averages of first moment m_t and second raw moment v_t of gradients; bias correction yields $\hat{m}_t, \hat{v}_t$; update uses $\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$. [1] Borrowed from prior work: combines the sparse-gradient adaptivity associated with AdaGrad and the non-stationary/online behavior associated with RMSProp, per the authors’ motivation. [1] Added/changed: explicit bias correction for initialization at zero and a careful stepsize choice that yields approximately bounded update magnitudes and invariance to gradient rescaling. [1] Historical importance: positioned as a versatile, memory-light first-order method that performs well across models/datasets and clarifies connections among adaptive methods. [1]

AdamW. Core idea: decouple weight decay from the gradient-based adaptive update, so that the decay term is not scaled by adaptive preconditioners. [2] Borrowed: starts from SGD-with-momentum and Adam-style adaptive gradients, and uses scheduling multipliers to handle learning-rate/decay schedules. [2] Added/changed: emphasizes that the equivalence between L_2 regularization and weight decay requires coupling λ to α for SGD, and does not hold for adaptive methods; proposes decoupled updates to reduce this coupling. [2] Why it mattered: reframes “weight decay” implementation details as a primary factor affecting regularization behavior under adaptive optimizers. [2]

SignSGD. Core idea: replace each gradient component by its sign, yielding 1-bit updates; analyze non-convex convergence under assumptions adapted to the ℓ_1 -geometry induced by signed steps. [3] Borrowed: builds on prior interest in sign-based methods (e.g., RPROP lineage as mentioned by the authors) and on SGD-style stochastic oracle frameworks, while explicitly adapting assumptions for signed updates. [3] Added/changed: establishes theoretical convergence guarantees for biased sign gradients, and develops a distributed “majority vote” scheme enabling 1-bit compression in both directions. [3] Why it mattered: provides a theory-backed route to communication-efficient training and a framework to reason about sign-based methods’ empirical performance. [3]

Shampoo. Core idea: structured preconditioning for tensors via Kronecker-factored accumulators, yielding updates equivalent to full-matrix preconditioned steps on vectorized parameters but with tractable structure. [4] Borrowed: uses online mirror descent / adaptive regularization tools, framing updates as preconditioned gradient steps. [4] Added/changed: introduces left/right preconditioners for matrix parameters (and generalizations to tensors) with regret bounds that achieve optimal $\mathcal{O}(\sqrt{T})$ scaling under stated conditions. [4] Why it mattered: offers a path toward second-order-like conditioning with analyzable guarantees while exploiting tensor structure. [4]

Muon. Core idea: for 2D weight matrices, replace raw gradients by an iteratively orthogonalized matrix $\text{Ortho}(G)$ computed via a Newton–Schulz-based iteration, then apply a momentum update using this transformed direction. [5] Borrowed: uses a momentum-style state variable and (per the source) targets hidden-layer weight matrices while recommending AdamW for embeddings/scalars, reflecting a hybrid design. [5] Added/changed: introduces a specific orthogonalization operator $\text{Ortho}(\cdot)$ (as defined in the source) and a practical implementation approach claimed to incur low overhead in typical settings. [5] Why it mattered historically (within the provided sources): it is presented as a specialized optimizer for hidden layers with empirically motivated design choices and a concrete integration recipe with existing training stacks. [5]

4 Optimizer Families

4.1 SGD & Momentum

SGD and SGD-with-momentum appear as baselines and as reference implementations whose hyper-parameters can be “lifted” into other methods in the provided sources. [3, 2, 1] AdamW provides explicit pseudo-code for SGD with momentum both with L_2 regularization and with decoupled weight decay (SGDW), highlighting how momentum and weight decay interact in standard training loops. [2] SignSGD describes momentum methods as taking steps according to a running average of recent gradients and uses this idea to define SIGNUM. [3] Beyond these descriptions, detailed momentum variants (e.g., Nesterov vs. Polyak) are *not specified in the provided sources*.

4.2 Adaptive methods

Adam. Adam maintains exponentially decaying averages of gradients and squared gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (2)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (3)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (4)$$

All operations are elementwise on vectors. [1] The authors emphasize bias correction due to zero initialization, invariance of update magnitudes to rescaling of the gradient, and approximate bounds on effective step magnitudes controlled by α . [1]

AdamW. AdamW argues that implementing weight decay as an L_2 penalty inside the gradient update ties the best regularization setting to the learning rate for SGD and breaks equivalence for adaptive gradient methods due to gradient scaling. [2] The proposed fix is to apply weight decay as a separate decay step (possibly schedule-scaled), rather than including $\lambda\theta$ in the gradient that is subsequently adaptively scaled. [2] The paper also describes a normalized weight decay and restart-based variants (AdamWR), but details beyond what is explicitly presented are *not specified in the provided sources*. [2]

4.3 Second-order / preconditioning

Shampoo. Shampoo frames updates as preconditioned steps, noting that when $W = \mathbb{R}^d$, an online mirror descent update with preconditioner H_t yields a preconditioned gradient step $w_{t+1} = w_t - \eta H_t^{-1} g_t$. [4] For matrix parameters $W_t \in \mathbb{R}^{m \times n}$ with gradient G_t , Shampoo uses left/right accumulators (e.g., $L_t = \epsilon I_m + \sum_{s \leq t} G_s G_s^\top$ and $R_t = \epsilon I_n + \sum_{s \leq t} G_s^\top G_s$) and updates of the form

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}, \quad (5)$$

with analysis connecting this to a single Kronecker preconditioner on $\text{vec}(W_t)$. [4]

4.4 Sign-based / communication-efficient methods

SignSGD and distributed majority vote. SignSGD uses updates of the form $x_{k+1} = x_k - \eta \text{sign}(\tilde{g})$, where $\tilde{g}$ is a stochastic gradient estimate, and motivates this as 1-bit compression of

gradient information. [3] In distributed training, the paper proposes majority vote aggregation of per-worker sign vectors, enabling 1-bit communication both worker→server and server→worker. [3] The analysis provides non-convex convergence rates for SignSGD (Theorem 1) and for distributed majority vote (Theorem 2) under assumptions including (in some regimes) unimodal and symmetric gradient noise. [3]

4.5 Specialized scaling- or structure-oriented methods

Muon (hidden-layer matrix optimizer). Muon is presented as an optimizer for hidden-layer weight matrices: it transforms a matrix gradient G into a semi-orthogonal update $\text{Ortho}(G)$ using an iterative Newton–Schulz-based procedure, then applies a momentum-style update. [5] The source provides a concrete algorithmic definition of the Newton–Schulz iteration used (including the fixed five-step “NewtonSchulz5” polynomial iteration) and describes how Muon integrates with AdamW for other parameter types in practical training. [5]

MuP (context, not an optimizer). MuP (Maximal Update Parametrization) aims to make learning-rate and related hyperparameters transferable across width, contrasting with parameterizations in which stable learning rates shrink with width. [6] The paper discusses “ μ P-compatible” learning-rate choices and includes guidance indicating that some techniques (e.g., weight decay in μ P Adam) may not be compatible as stated. [6] Since MuP does not define a new update rule, we treat it as a scaling framework that conditions the behavior of optimizers such as SGD and Adam rather than a distinct optimizer family. [6]

5 Comparative Analysis

5.1 Side-by-side update equations (consistent notation)

We collect update rules in a unified notation. For optimizers whose full details are not specified in the provided sources (e.g., exact SGD momentum variant choices), we state only what is specified. [2, 3]

SGD (baseline form).

$$\theta_t = \theta_{t-1} - \alpha g_t, \quad (6)$$

used as a baseline comparator throughout the sources. [1, 3, 4]

Adam.

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}, \quad (7)$$

with $(m_t, v_t, \hat{m}_t, \hat{v}_t)$ as defined earlier. [1]

AdamW (decoupled decay form). In AdamW, weight decay is applied separately from the gradient-based adaptive update; the key claim is that this decoupling removes an explicit coupling between λ and α (and avoids scaling the decay term by Adam’s adaptive preconditioner). [2] The exact line-by-line AdamW pseudo-code and its schedule multiplier η_t are given in the source. [2]

SignSGD.

$$x_{k+1} = x_k - \eta \operatorname{sign}(\tilde{g}_k), \quad (8)$$

and in distributed majority vote:

$$x_{k+1} = x_k - \eta \operatorname{sign}\left(\sum_{m=1}^M \operatorname{sign}(\tilde{g}_k^{(m)})\right). \quad (9)$$

[3]

Shampoo (matrix form).

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}. \quad (10)$$

[4]

Muon (matrix form, hidden layers). Muon uses a momentum accumulator V_t updated using the orthogonalized gradient direction $\operatorname{Ortho}(G_t)$, where $\operatorname{Ortho}(\cdot)$ is computed by a fixed-step Newton–Schulz iteration over a scaled gradient matrix; the precise operator and iteration are defined in the source. [5] A minimal schematic consistent with the source is:

$$\tilde{G}_t \leftarrow \operatorname{Ortho}(G_t), \quad (11)$$

$$V_t \leftarrow \mu V_{t-1} + \tilde{G}_t, \quad (12)$$

$$W_t \leftarrow W_{t-1} - \eta V_t, \quad (13)$$

with practical details (e.g., application to 2D parameters and integration with AdamW elsewhere) as specified by the source. [5]

5.2 Table 1: Mechanism-level comparison

Optimizer	State variables	Core mechanism	Extra compute	Extra memory
SGD	none (baseline)	$-\alpha g_t$ step [1, 3]	low [1]	low [1]
Adam	m_t, v_t (+ bias correction) [1]	adaptive per-parameter step via $\hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$; update magnitude approx. bounded, invariant to gradient rescaling [1]	elementwise ops [1]	two parameter-sized buffers [1]
AdamW	Adam-style state + separate decay [2]	decoupled weight decay step; avoids scaling decay by adaptive gradients and reduces explicit λ - α coupling [2]	small (extra decay op) [2]	Adam-like [2]
SignSGD	none required (core) [3]	1-bit sign update; distributed majority vote compresses both directions [3]	sign + aggregation [3]	low [3]
Shampoo	preconditioners (e.g., L_t, R_t) [4]	Kronecker-structured matrix/tensor preconditioning; equivalent to a single preconditioner on vec form [4]	matrix roots/inverses [4]	$m^2 + n^2$ for $m \times n$ (implied by storing L_t, R_t) [4]
Muon	momentum V_t + orthogonalization procedure [5]	replace G by $\text{Ortho}(G)$ via fixed Newton-Schulz iteration; apply momentum and update hidden-layer matrices [5]	iterative matrix ops [5]	momentum buffer + temporaries [5]

Table 2: Mechanism-level comparison grounded in algorithmic descriptions in the provided sources. [1, 2, 3, 4, 5]

5.3 Table 2: Practical tradeoffs

Optimizer	Hyperparameter sensitivity	Stability / failure modes	Typical regimes and cautions
SGD	baseline; detailed sensitivity <i>not specified</i> [3]	detailed failure modes <i>not specified</i> [1]	serves as comparator; SIGNUM suggests many settings can be lifted from SGD when tuned [3]
Adam	authors report good defaults and robustness across tasks [1]	bias correction important when β_2 close to 1; removing it can cause early instabilities [1]	designed for sparse gradients and non-stationary objectives; claims invariance to gradient rescaling [1]
AdamW	aims to decouple learning rate and decay; argues adaptive scaling makes L_2 behave differently than true decay [2]	failure mode: coupling λ to α in L_2 -as-decay and adaptive scaling of regularizer gradients [2]	recommended when using adaptive methods with weight decay-like regularization [2]
SignSGD	fewer continuous hyperparameters; distributed majority vote adds worker count effects [3]	theory depends on assumptions tailored to signed updates and (for improved rate) unimodal symmetric noise [3]	especially efficient in problems with dense gradients/noise and ℓ_1 -geometry; can match SGD rates in practice per experiments [3]
Shampoo	practical heuristics (e.g., projection omission) and computational expense noted [4]	projection step can be expensive and is often omitted; regret analysis assumes conditions (e.g., rank constraints in stated theorem) [4]	targets tensor parameters; uses Kronecker structure to approximate full preconditioning [4]
Muon	specialized; requires 2D parameters for orthogonalization operator [5]	cautions: applies to hidden-layer matrices; recommends AdamW for embeddings/scalars and other parameters [5]	intended for hidden layers; implementation uses fixed Newton–Schulz steps (“NewtonSchulz5”) [5]

Table 3: Practical tradeoffs and cautions explicitly discussed in the provided sources; items marked “not specified” are omitted by the sources and are not inferred here. [1, 2, 3, 4, 5]

5.4 Failure modes and “performs poorly” contexts

Adam: bias correction and sparse-gradient settings. The Adam paper reports that removing bias correction yields a RMSProp-with-momentum-like variant and observes instability for β_2 close to 1 in early epochs without bias correction, especially when sparse-gradient robustness motivates β_2 near 1. [1] Beyond this, contexts where Adam performs poorly are *not specified in the provided sources*.

AdamW: regularization coupling in adaptive methods. AdamW emphasizes that the common equivalence between L_2 regularization and weight decay does not hold for adaptive gradient methods, because the regularizer gradient is scaled; consequently, some parameters may be effec-

tively regularized less under Adam+ L_2 than under decoupled decay. [2] Additional failure modes are *not specified in the provided sources*.

SignSGD: geometry and noise assumptions. SignSGD’s theoretical rates rely on assumptions adapted to signed updates and discuss regimes where dense gradients/noise and ℓ_1 -geometry yield favorable dimension dependence; improved distributed rates additionally use unimodal symmetric noise assumptions. [3] Performance outside these regimes is *not specified in the provided sources*.

Shampoo: computational and projection considerations. Shampoo notes that projections required to keep iterates within bounded domains can be computationally expensive and are rarely performed in practice (and thus omitted in the algorithm for a looser bound). [4] The computational cost of preconditioner operations is an inherent practical consideration. [4]

Muon: scope restrictions. Muon is explicitly targeted to 2D weight matrices (hidden layers) and recommends using AdamW for embeddings/scalars and other parameter types, indicating scope limitations as a practical “performs poorly” or “not applicable” context. [5]

MuP context: compatibility caveats. MuP includes explicit guidance that certain practices (e.g., weight decay in μ P Adam) are not compatible as stated, which can be interpreted as a failure mode if naively transferred. [6]

6 Research Insights & Open Questions

6.1 Synthesized insights (grounded in provided sources)

1. Bias correction is not a cosmetic detail: when decay rates (especially β_2) are close to 1, initialization bias can materially destabilize training without correction in early epochs. [1]
2. Decoupling weight decay from adaptive gradient scaling is conceptually important because adaptive methods scale gradients by historical magnitudes, which can unintentionally scale regularizer gradients and alter the intended regularization strength across parameters. [2]
3. Signed updates induce a natural ℓ_1 -geometry, and analyzing SignSGD requires assumptions finer-grained than standard SGD analyses; under these assumptions, SignSGD achieves a non-convex rate with favorable dimension dependence in dense-gradient regimes. [3]
4. Majority-vote aggregation enables 1-bit communication in both directions and can match the theoretical speedup of distributed SGD under assumptions validated empirically (including approximate symmetry/unimodality of noise in deep learning gradients). [3]
5. Shampoo shows how tensor structure can yield preconditioning that is equivalent (on vectorized parameters) to applying a Kronecker product preconditioner, enabling regret guarantees while avoiding full dense preconditioning. [4]
6. Shampoo’s regret bounds can achieve the optimal $\mathcal{O}(\sqrt{T})$ scaling under stated conditions (e.g., rank constraints), connecting structured preconditioning to best-possible stochastic/online rates. [4]

7. Muon’s design highlights a different kind of structure: for weight matrices, constraining updates toward semi-orthogonality (via Newton–Schulz iterations) is proposed as a useful inductive bias for hidden-layer optimization. [5]
8. Hybridization is an explicit design pattern in the provided sources: AdamW proposes mixing adaptive updates with decoupled decay; Muon proposes using Muon on some parameter subsets and AdamW elsewhere. [2, 5]
9. Scaling behavior can hinge on parameterization as much as on the optimizer: MuP aims to preserve update magnitudes across width so that learning rates remain stable and transferable. [6]
10. Some “best practices” do not transfer unchanged across scaling frameworks: MuP explicitly warns about incompatibility of certain regularization choices (e.g., weight decay in μ P Adam) in its setting. [6]

6.2 Open questions (motivated directly by provided sources)

1. Can the decoupled-regularization viewpoint of AdamW be generalized into a principled taxonomy of which terms should (or should not) be passed through adaptive preconditioners? [2]
2. SignSGD’s improved distributed theory uses unimodal symmetric noise assumptions; how robust are these guarantees under heavier-tailed or multimodal gradient noise observed outside the examined regimes? [3]
3. What are the sharp boundaries of the “dense gradients/noise” regime in which SignSGD enjoys favorable dimension dependence, and how can practitioners diagnose this regime online? [3]
4. Shampoo uses Kronecker-structured accumulators; what structure classes beyond Kronecker products yield similar guarantees with manageable compute for modern architectures? [4]
5. Shampoo’s analysis includes conditions such as rank constraints in a main theorem; when and why do these conditions approximately hold in deep networks, and can they be enforced or encouraged? [4]
6. Muon proposes a fixed-step Newton–Schulz procedure (“NewtonSchulz5”); how sensitive is performance to the polynomial/iteration choice, and can stability be characterized theoretically? [5]
7. Muon is targeted to hidden-layer matrices and recommends AdamW for other parameters; can one unify such heterogeneous parameter-type rules into a single optimizer with analyzable behavior? [5]
8. MuP warns about incompatibilities such as weight decay in μ P Adam; what are the minimal modifications to recover regularization effects while preserving MuP’s hyperparameter transfer properties? [6]
9. Across these methods, how should one compare “compute budgets” fairly when some methods trade extra per-step compute (Shampoo, Muon) for fewer steps or different stability behavior? [4, 5]

10. The provided sources emphasize different geometries (elementwise adaptivity, ℓ_1 sign geometry, Kronecker preconditioning, orthogonalized directions); is there a unifying geometric framework that predicts when each geometry is beneficial? [1, 3, 4, 5]

7 Conclusion

The provided sources illustrate a coherent optimizer lineage in which each method targets a specific bottleneck: Adam focuses on adaptive scaling via moment estimates and bias correction with bounded, rescaling-invariant updates. [1] AdamW reframes regularization as an algorithmic design choice, arguing that decoupling weight decay from adaptive scaling reduces implicit hyperparameter coupling and changes the effective regularization behavior of adaptive methods. [2] SignSGD shifts attention to communication and robustness under extreme quantization, providing non-convex convergence theory under assumptions suited to signed updates and offering a majority-vote protocol for 1-bit distributed training. [3] Shampoo pushes toward structured second-order information via Kronecker preconditioning with regret guarantees, while Muon explores an alternative structure for hidden-layer matrices by orthogonalizing update directions via Newton–Schulz iterations and pairing this with momentum. [4, 5]

Finally, MuP underscores that optimizer behavior is intertwined with parameterization and scaling: stable learning-rate transfer across width may require changing the model’s parametrization rather than the optimizer rule itself. [6] Taken together, these works suggest that “optimizer design” is best viewed as designing *update geometry* under practical constraints—compute, memory, communication, and scaling—with careful attention to which algorithmic details (e.g., bias correction, decoupled decay, or parameterization) fundamentally change the training dynamics. [1, 2, 3, 4, 6]

A Update-rule cheat sheet

Optimizer	Cheat-sheet update
SGD	$\theta_t \leftarrow \theta_{t-1} - \alpha g_t$ (baseline form) [1, 3]
Adam	$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$, $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$, $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$, $\theta_t \leftarrow \theta_{t-1} - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ [1]
AdamW	Adam-style adaptive step plus separate weight decay step (decoupled from adaptive scaling), with schedule multiplier η_t in pseudo-code. [2]
SignSGD	$x_{k+1} \leftarrow x_k - \eta \text{sign}(\tilde{g}_k)$; distributed: $x_{k+1} \leftarrow x_k - \eta \text{sign}(\sum_m \text{sign}(\tilde{g}_k^{(m)}))$ [3]
Shampoo	$W_{t+1} \leftarrow W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}$ with $L_t = \epsilon I_m + \sum_{s \leq t} G_s G_s^\top$, $R_t = \epsilon I_n + \sum_{s \leq t} G_s^\top G_s$ [4]
Muon	For 2D matrices: $\tilde{G} \leftarrow \text{Ortho}(G)$ via fixed Newton–Schulz steps; momentum update $V \leftarrow \mu V + \tilde{G}$; $W \leftarrow W - \eta V$ (integration details in source). [5]

Table 4: Update-rule cheat sheet. Items are stated only to the extent specified in the provided sources. [1, 2, 3, 4, 5]

B Glossary of terms

Bias correction Adjustment of moment estimates initialized at zero to reduce early-iteration bias, as used in Adam via division by $(1 - \beta^t)$. [1]

Decoupled weight decay Applying a multiplicative parameter decay step separately from the gradient-based update, advocated to avoid adaptive scaling of the regularizer gradient. [2]

Majority vote (SignSGD) Distributed aggregation of worker sign vectors by taking the sign of the sum of signs, enabling 1-bit communication in both directions. [3]

Preconditioning Multiplying gradients by (an approximation to) an inverse curvature-like matrix to change the update geometry; Shampoo uses structured (Kronecker) preconditioners. [4]

Kronecker structure Representing a large matrix as a Kronecker product of smaller matrices; Shampoo connects left/right matrix preconditioners to a Kronecker product preconditioner on vectorized parameters. [4]

Orthogonalized update (Muon) Transforming a matrix gradient into a semi-orthogonal matrix $\text{Ortho}(G)$ using an iterative Newton–Schulz procedure before applying a momentum update. [5]

MuP A parameterization intended to stabilize and transfer hyperparameters across width scaling; not itself an optimizer update rule. [6]

References

- [1] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. Published as a conference paper at ICLR 2015. [1]
- [2] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. Published as a conference paper at ICLR 2019. [2]
- [3] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Anima Anandkumar. signSGD: Compressed Optimisation for Non-Convex Problems. ICML 2018 (as provided). [3]
- [4] Vineet Gupta, Timon Gasthaus, and others (as listed in the provided source). Shampoo: Preconditioned Stochastic Tensor Optimization. ICML 2018 (as provided). [4]
- [5] Keller Jordan and collaborators (as listed in the provided source). Muon: An optimizer for hidden layers in neural networks. Blog PDF as provided (2024). [5]
- [6] Greg Yang and collaborators (as listed in the provided source). Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer (MuP / Maximal Update Parametrization). As provided in `MuP.pdf`. [6]